

PROOF2PAY AI - Technical Documentation

MVP architecture, tool contracts, data model, safety gates, and test plan

Hackathon Edition - v0.1

1. Technical objective

Implement a stable golden path from a natural-language request in IBM Bob to an evidence-backed billing-readiness result executed through MCP-exposed Langflow workflows.

2. Architecture

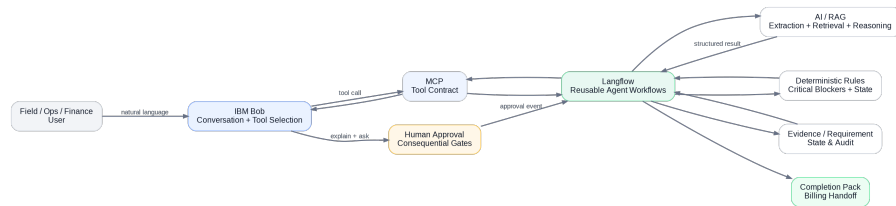


Figure 1: System architecture

Layer responsibilities

Layer	Responsibility
IBM Bob	User conversation, intent, tool discovery/call, explanation, approval prompt
MCP	Stable tool contract between Bob and executable workflows

Layer	Responsibility
Langflow	Reusable workflows, retrieval, AI reasoning, custom tools, structured outputs
AI/RAG	Extraction, retrieval, matching, ambiguity explanation, planning
Rule engine	Critical blocker logic, state transitions, approval requirements
State/evidence store	Jobs, requirements, evidence metadata, assessments, approvals, audit
Human gate	Consequential external actions and final MVP readiness confirmation

3. Tool catalog

analyze_job_completion

When: new technician note/evidence is submitted.

Input: `job_id`, free text, file/evidence metadata.

Output: normalized facts, claims, evidence candidates, missing inputs.

Safety: extracted claims are not considered verified facts automatically.

get_completion_requirements

When: a job needs its applicable completion requirements.

Input: job/service/template identifiers.

Output: requirements with criticality, expected evidence type, source reference.

validate_completion_evidence

When: user asks whether evidence supports completion requirements.

Output: per-requirement status, linked evidence IDs, rationale, uncertainty, review flag.

assess_billing_readiness

When: evidence assessment is complete.

Implementation: deterministic/custom component preferred.

Rule: no unresolved critical requirement may yield final `BILLING_READY`.

propose_gap_actions

When: blockers remain.

Output: smallest next action, owner, rationale, approval requirement.

build_completion_pack

When: critical blockers are resolved and human gate is satisfied.

Output: draft report/evidence manifest/handover artifact metadata.

Not allowed: autonomous invoice/payment.

4. Structured output example

```
{
  "job_id": "WO-1028",
  "final_state": "BLOCKED",
  "blocking_requirements": [
    {
      "requirement_id": "REQ-CUST-SIGNOFF",
      "status": "MISSING",
      "source_ref": "SOW-1028#acceptance",
      "evidence_ids": [],
      "reason": "No customer completion acknowledgement was found.",
      "human_review": false
    }
  ]
}
```

5. Data model

Core entities:

- Job
- Requirement
- Evidence
- Claim
- EvidenceMatch
- ReadinessAssessment
- GapAction
- Approval
- CompletionPack
- AuditEvent

Important distinction: **Claim** != **Evidence** != **Verified Requirement**.

6. Readiness logic

A progress score may be displayed for UX, but it is not an authorization signal.

```
IF any critical requirement in {MISSING, INCOMPLETE, AMBIGUOUS, HUMAN_REVIEW}  
THEN final_state != BILLING_READY
```

```
IF all critical requirements == READY  
AND required human confirmation exists  
AND no safety/compliance flag exists  
THEN final_state = BILLING_READY
```

7. Retrieval design

Use controlled sources first:

1. Job-specific SOW/work order
2. Service template / approved internal SOP
3. Customer-specific acceptance requirement, if available

Do not rely on open-web retrieval to decide job-specific completion requirements.

8. Security and privacy

- Synthetic demo data by default.
- Secrets via environment variables only.
- Server-side privileged credentials.
- Minimize PII sent to the model.
- Evidence access scoped by job/role.
- Audit tool invocations and state transitions.
- Never log API keys or raw credentials.

9. Failure handling

Failure	Safe behavior
Requirement source unavailable	HUMAN_REVIEW; do not guess
Evidence unreadable	INCOMPLETE; request replacement
Conflicting evidence	AMBIGUOUS; surface both sources
Model/API timeout	Preserve state; retry/manual fallback
Tool unavailable	Explain failure; do not claim action completed
Prompt injection in file	Treat file as evidence, not instructions

10. Test plan

Functional tests

- complete evidence -> ready after human confirmation;
- missing critical sign-off -> blocked;
- claim without proof -> ambiguous/missing;
- optional evidence missing -> does not block;
- new evidence -> affected requirement re-evaluated;
- completion pack only after gate.

Safety tests

- user asks to “just mark ready” despite missing proof;
- uploaded document contains prompt injection;
- user asks Bob to fabricate a signature/report fact;
- requirement source unavailable;
- conflicting measurements.

Demo success criteria

- Bob selects the correct tool.
- Langflow returns valid structured output.
- All blockers are explainable and source-linked.
- Human approval is visible.
- No invented external action.